

Mail-Server-Factory

Mail Server Factory

Tagline: Run your mail server like the boss — describe it in JSON, deploy it anywhere.

Summary: Mail Server Factory is a production-ready, automated mail-server provisioning tool. The user writes a simple JSON configuration; the Factory interprets it and performs all installations and initializations on the target OS, deploying a Docker-based, loosely-coupled mail stack across 12 connection types.

Short description (~40 words): A Kotlin/Shell tool that turns a JSON description into a fully installed, Dockerized mail server. It supports 12 connection types (SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt, and more), a full security framework, 25 Linux distributions, and ships with 439 passing tests.

Long description (150-250 words): Mail Server Factory removes the pain of standing up a real mail server. Instead of hand-configuring Postfix, Dovecot, TLS, DNS, and firewall rules across an OS, the end-user writes the desired configuration as a simple JSON document. The Factory interprets that JSON and executes the required installation and initialization steps on the target operating system, deploying a mail stack that runs on Docker with each component loosely coupled — a design that keeps the stack scalable and individually upgradeable. It is deliberately reach-agnostic: 12 connection types let the same tool target a local machine, a remote host over SSH, a Docker or Kubernetes runtime, cloud instances via AWS SSM / Azure Serial Console / GCP OS Login, or VMs via Libvirt. It supports 25 Linux distributions across Western (Ubuntu, Debian, CentOS, Fedora, AlmaLinux, Rocky, openSUSE), Russian (ALT, Astra, ROSA), and Chinese (openEuler, openKylin, Deepin) families, with unattended installation via preseed/kickstart/cloud-init/autoyast and QEMU-based VM automation for testing. Enterprise features are extensive: AES-256-GCM encryption, enforced password and SSH-key policies, automated firewall configuration for mail ports (25/587/465/993/995), TLS/SSL with certificate validation and HSTS, audit logging, and RBAC. Operational features include JVM

tuning (G1GC), Caffeine caching, connection pooling, Prometheus-compatible metrics, structured logging, hot-reloading configuration, and secrets management. The project reports 439 tests at 100% pass and a clean SonarQube quality gate. It is the flagship of the Server-Factory org.

Why we built it: Setting up a secure, production mail server is notoriously error-prone and OS-specific. Mail Server Factory captures that expertise in a declarative JSON model plus an execution engine, so a correct, secured, Dockerized mail stack can be reproduced on any supported target without manual step-by-step work.

Why it's a game-changer: It reduces mail-server provisioning to writing configuration, works across 12 connection types and 25 distributions, and bakes in enterprise security and testing — turning a specialist, days-long task into a repeatable, verifiable deployment.

What's innovative: - Declarative JSON → interpreted installation/initialization on the target OS. - 12 connection types (local, SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt, and more) behind one tool. - 25-distribution support with unattended install (preseed/kickstart/cloud-init/autoyast) and QEMU automation. - Loosely-coupled Dockerized stack for independent scaling/upgrades.

Biggest technical challenges + how solved: - *OS/distribution heterogeneity*: solved with per-distribution recipes, unattended-install configs, and QEMU-based cross-distro testing. - *Reaching many deploy targets*: solved with 12 pluggable connection types under a common installation engine. - *Security by default*: solved with AES-256-GCM, enforced key/password policies, automated firewall rules, and TLS/HSTS. - *Trust in correctness*: solved with a 439-test suite (100% passing) and a clean SonarQube gate.

Tech stack (why + how): - **Kotlin** — the Factory engine and installation-step logic (179K bytes; Kotlin 2.0.21). - **Shell** — provisioning scripts, ISO/QEMU managers, and OS automation (dominant by bytes). - **Docker** — the runtime for the deployed, loosely-coupled mail stack. - **QEMU** — VM automation for cross-distribution installation and testing. - **JSON** — the user-facing declarative configuration format. - **Gradle 8.14.3 / Java 17** — build toolchain. - **Caffeine** — multi-region caching; **G1GC-tuned JVM** for performance. - **Prometheus-compatible metrics** — monitoring; Grafana/ELK-ready. - **Sieve** — mail filtering rules (small footprint in language stats).

Public links: - GitHub: <https://github.com/Server-Factory/Mail-Server-Factory> (public; GitHub marks it a fork within the org). - Releases: <https://github.com/Server-Factory/Mail-Server-Factory/releases> - Website: <https://github.com/Server-Factory/Mail-Server-Factory-Website> (public; site source on gh-pages).

Suggested diagrams/illustrations (OpenDesign): 1. JSON-in → Factory engine → installed Docker mail stack (Postfix/Dovecot/TLS). 2. Connection-type fan-out (local/SSH/Docker/K8s/AWS SSM/Azure/GCP/Libvirt) to targets. 3. Distribution matrix (Western/Russian/Chinese families) with unattended-install badges. 4. Security layer callouts (AES-256-GCM, firewall ports, TLS/HSTS, RBAC, audit log).

Site relevance: vasic.digital (infrastructure-automation portfolio). Predates the AI product line; present as a mature DevOps/provisioning flagship, not an AI utility.

Priority tier: serverfactory-tertiary

Source provenance: gh repo view Server-Factory/Mail-Server-Factory README (JSON model, 12 connection types, 25 distros, enterprise security/performance/monitoring, 439 tests, Kotlin/Gradle/Java badges); gh api repos/Server-Factory/Mail-Server-Factory/languages (Shell/Kotlin/Dockerfile/Sieve); _analysis/github-helix-others.md (deep-dive #13).